Adder

2023/11/03

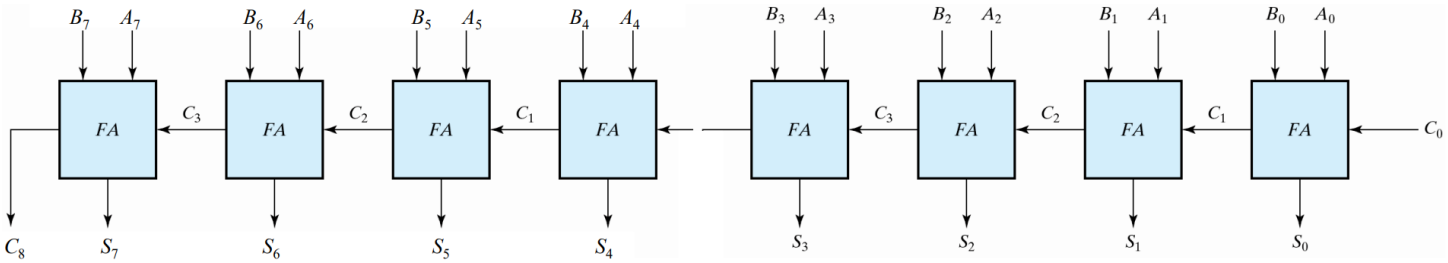
B113040045 許育菖

實驗內容

Exercise1

題目說明

Design and verify the 8-bit ripple-carry adder composed of eight full-adders by using **Verilog Structural level modeling**.



推導

Half-Adder:

```
`timescale 1ns / 1ps

module halfAdder(output S, C, input x, y);
    xor(S, x, y);
    and(C, x, y);
endmodule
```

Full-Adder:

```
`timescale 1ns / 1ps

module fullAdder(output S, C, input x, y, z);
    wire S1, C1, C2;
```

```
        halfAdder HA1 (S1, C1, x, y);
        halfAdder HA2 (S, C2, S1, z);

        or(C, C2, C1);
    endmodule
```

Verilog code:

```
    `timescale 1ns / 1ps
    module exercise1(output[7:0] Sum, output C8, input [7:0] A, B, input C0);
        wire C1, C2, C3, C4, C5, C6, C7;
        fullAdder       FA0 (Sum[0], C1, A[0], B[0], C0),
                        FA1 (Sum[1], C2, A[1], B[1], C1),
                        FA2 (Sum[2], C3, A[2], B[2], C2),
                        FA3 (Sum[3], C4, A[3], B[3], C3),
                        FA4 (Sum[4], C5, A[4], B[4], C4),
                        FA5 (Sum[5], C6, A[5], B[5], C5),
                        FA6 (Sum[6], C7, A[6], B[6], C6),
                        FA7 (Sum[7], C8, A[7], B[7], C7);

    endmodule
```

Half-Adder: $S = x \oplus y$

$C = xy$

Full-Adder: $S_i = (x \oplus y) \oplus C_{i-1}$

$C_i = (x \oplus y)z + xy$

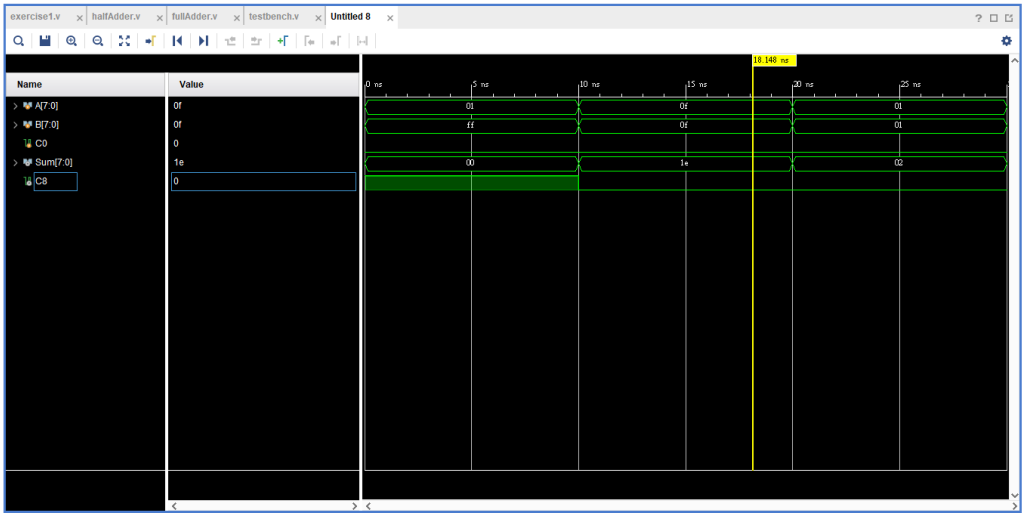
8-bit ripple-carry = 8 個 Full-Adder 連接起來

$$S_i = (x \oplus y) \oplus C_{i-1}$$

$$C_i = (x \oplus y)z + xy$$

$$i \in N,\ 0 < i < 8$$

結論



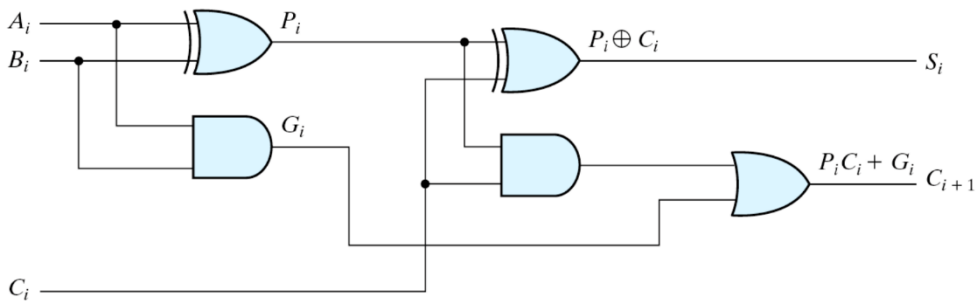
Exercise2

題目說明

Design and verify the 4-bit carry-look ahead adder by using

Verilog Structural level modeling.

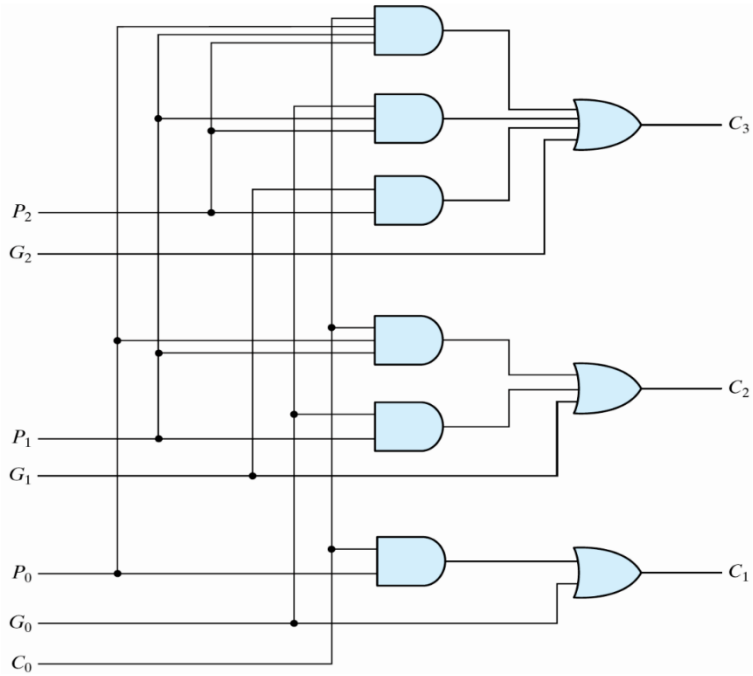
- carry propagate: $P_i = A_i \oplus B_i$, carry generate: $Gi = A_iB_i$
- sum: $S_i = P_i \oplus C_i$, carry: $C_{i+1} = G_i + P_iC_i$
- $C_1 = G_0+P_0C_0$
- $C_2 = G_1+P_1C_1 = G_1+P_1(G_0+P_0C_0) = G_1+P_1G_0+P_1P_0C_0$
- $C_3 = G_2+P_2C_2 = G_2+P_2G_1+P_2P_1G_0+ P_2P_1P_0C_0$
- $C_4 = G_3+P_3C_3 = \ldots$



推導

Since $C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$

$\Rightarrow$ $\mathbf{C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0}$

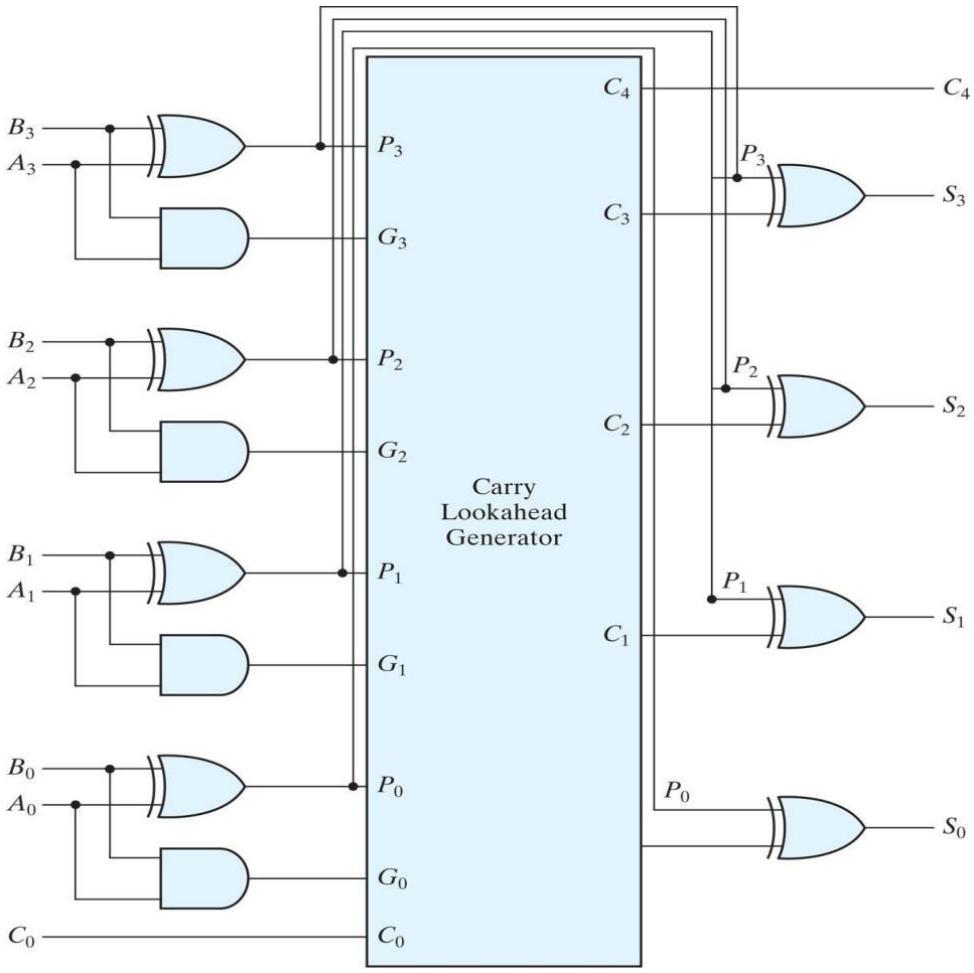


Verilog code:

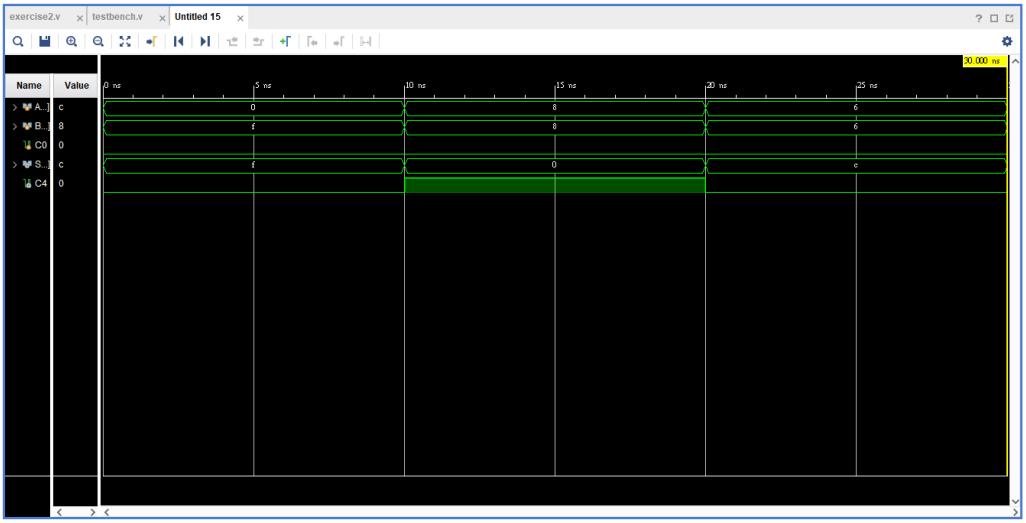
```
timescale 1ns / 1ps

module exercise2(output [3:0] Sum, input [3:0] A, B, output C4, input C0);
wire G0, G1, G2, G3;
and(G0, A[0], B[0]);
and(G1, A[1], B[1]);
and(G2, A[2], B[2]);
and(G3, A[3], B[3]);
```

```
wire P0, P1, P2, P3;
xor(P0, A[0], B[0]);
xor(P1, A[1], B[1]);
xor(P2, A[2], B[2]);
xor(P3, A[3], B[3]);

wire C1, C2, C3;
wire and0;
and(and0, P0, C0);
or(C1, G0, and0);

wire and1, and2;
and(and1, P1, G0);
and(and2, P1, P0, C0);
or(C2, G1, and1, and2);

wire and3, and4, and5;
and(and3, P2, G1);
and(and4, P2, P1, G0);
and(and5, P2, P1, P0, C0);
or(C3, G2, and3, and4, and5);

wire and6, and7, and8, and9;
and(and6, P3, G2);
and(and7, P3, P2, G1);
and(and8, P3, P2, P1, G0);
and(and9, P3, P2, P1, P0, C0);
or(C4, G3, and6, and7, and8, and9);

/*
wire and10, and11, and12, and13, and14;
and(and10, P4, G3);
and(and11, P4, P3, G2);
and(and12, P4, P3, P2, G1);
and(and13, P4, P3, P2, P1, G0);
and(and14, P4, P3, P2, P1, P0, C0);
or(C4, G4, and10, and11, and12, and13, and14);
*/
```

```
xor(Sum[0], P0, C0);
xor(Sum[1], P1, C1);
xor(Sum[2], P2, C2);
xor(Sum[3], P3, C3);

endmodule
```



結論



實驗心得

本週的實驗內容為製作加法器。加法器主要分為兩種，一種是 ripple-carry adder，另一種則是 carry-look ahead adder，兩種的差別在於：ripple adder 在進行多位數的加法時，會比 look ahead adder 花費更多的時間，因為 look ahead adder 是直接從傳入值得出每個位數加起來的結果，而 ripple adder 在計算當前位數時，需要先求出前一個位數的值，並將前一位數相加後的結果帶入運算。

雖然說 look ahead adder 在運算時可以比較節省時間，但在撰寫它的程式邏輯時好麻煩喔，程式碼要寫一大串，果然這世界上沒有甚麼事情是兩全其美的。